

GestorCan

Documento tecnico - Clientes, mini frontend y PDFs con branding de residencia

Version 2 - Primera pantalla real: listado de clientes, ficha, endpoint PDF y personalizacion por residencia.

Objetivo: dejar documentado el avance desde el backend base hasta una funcionalidad visible y exportable, preparada para convertirse en plantilla reutilizable para facturas, listados e informes.

Elemento	Estado actual
Backend local	FastAPI ejecutado con uvicorn app.main:app --reload --port 8080
Modulo principal trabajado	Clientes
Documento PDF	Ficha de cliente con cabecera tipo factura y branding de residencia
Base SaaS	Separacion entre marca GestorCan y marca de la residencia cliente

1. Resumen del avance

En esta fase se ha pasado de un backend funcional a una primera funcionalidad real visible: listado de clientes, pantalla de ficha y generacion de PDF desde el endpoint de cliente.

La decision importante de arquitectura ha sido separar la marca del software, GestorCan, de la marca operativa de cada residencia. GestorCan queda como identidad de la aplicacion; los documentos emitidos hacia clientes finales deben usar la imagen de la residencia.

Area	Decision	Motivo
Frontend inicial	HTML sencillo sin frameworks	Reducir complejidad y validar el flujo antes de incorporar React u otra capa.
PDF	ReportLab	Permite generar documentos desde el backend con datos reales.
Branding documental	Residencia cliente	Facturas, fichas e informes deben parecer emitidos por la residencia.
Branding de app	GestorCan	La marca del SaaS se mantiene en login, dashboard y panel interno.

2. Punto de partida documentado

La documentacion tecnica anterior dejaba descrita la base del backend con FastAPI, PostgreSQL en Railway, repositorio gestorcan-backend, endpoints iniciales y el criterio de avanzar por hitos pequeños y documentados. Esta version continua ese criterio sobre el primer modulo funcional de clientes.

El documento anterior tambien marcaba como siguiente objetivo convertir clientes en un CRUD real con schemas, get_db, POST y GET, y documentar el primer CRUD real de GestorCan.

3. Estructura trabajada

La estructura del proyecto ya se esta separando en carpetas de aplicacion, modelos, routers y recursos estaticos.

```
gestorcan-backend/
|-- app/
| |-- main.py
| |-- database.py
| |-- models/
| | |-- clientes.py
| | |-- mascotas.py
| | |-- residencias.py
| |-- routers/
| | |-- clientes.py
| | |-- residencias.py
| |-- schemas/
| |-- clientes.py
|-- static/
| |-- logos/
| |-- logo.png
|-- pdfs/
| |-- cliente_1.pdf
|-- requirements.txt
```

Archivo / carpeta	Funcion
app/routers/clientes.py	Endpoints de clientes, ficha, mascotas y PDF.
app/routers/residencias.py	Endpoint de prueba para crear/configurar residencia.
app/models/residencias.py	Modelo SQLAlchemy con datos de residencia, logo y colores.
static/logos/	Ubicacion de logos de residencias.
pdfs/	Salida local temporal para PDFs generados.

4. Router de clientes

El router de clientes debe conservar un unico APIRouter con prefix /clientes. Un error detectado durante el desarrollo fue crear un segundo router dentro del mismo archivo, lo que provocaba rutas no registradas o comportamientos confusos en Swagger.

```
router = APIRouter(
    prefix="/clientes",
    tags=["Clientes"]
)
```

Regla practica: un archivo de router debe tener un unico objeto router principal. No se debe volver a declarar router = APIRouter() al final del archivo.

5. Endpoints actuales de clientes

Endpoint	Funcion	Estado
GET /clientes/	Listar clientes	Operativo
POST /clientes/	Crear cliente	Operativo
GET /clientes/{cliente_id}	Obtener cliente por ID	Operativo
PUT /clientes/{cliente_id}	Actualizar cliente	Operativo
DELETE /clientes/{cliente_id}	Eliminar cliente	Operativo
GET /clientes/{cliente_id}/mascotas	Listar mascotas del cliente	Operativo
GET /clientes/{cliente_id}/ficha	Devolver cliente y mascotas	Operativo
GET /clientes/{cliente_id}/pdf	Generar ficha PDF	Operativo

6. Endpoint PDF

El endpoint PDF consulta el cliente, sus mascotas y la residencia configurada. Despues construye un PDF con cabecera tipo factura, logo, datos de cliente y bloque de mascotas.

```
@router.get("/{cliente_id}/pdf")
def generar_pdf_cliente(cliente_id: int, db: Session = Depends(get_db)):
    cliente = db.query(Cliente).filter(Cliente.id == cliente_id).first()

    if not cliente:
        raise HTTPException(status_code=404, detail="Cliente no encontrado")

    mascotas = db.query(Mascota).filter(Mascota.cliente_id == cliente_id).all()
    residencia = db.query(Residencia).order_by(Residencia.id.desc()).first()

    if not residencia:
        raise HTTPException(status_code=404, detail="No hay residencia configurada")

    ruta = f"pdfs/cliente_{cliente_id}.pdf"
    os.makedirs("pdfs", exist_ok=True)

    # Construcccion del PDF con ReportLab
    doc.build(contenido)

    return FileResponse(
        path=ruta,
        filename=f"cliente_{cliente_id}.pdf",
        media_type="application/pdf"
    )
```

Durante las pruebas se confirmo que la URL local <http://127.0.0.1:8080/clientes/1/pdf> devuelve el PDF correctamente.

7. Modelo Residencia

Se ha creado una tabla residencias para almacenar los datos visuales y fiscales de la residencia cliente. Esta tabla sera clave para personalizar facturas, listados, informes y fichas.

```
class Residencia(Base):
    __tablename__ = "residencias"

    id = Column(Integer, primary_key=True, index=True)
    nombre = Column(String, nullable=False)
    cif = Column(String)
    direccion = Column(String)
    telefono = Column(String)
    email = Column(String)
    web = Column(String)
    logo_url = Column(String)
    color_principal = Column(String, default="#3CB371")
    color_secundario = Column(String, default="#1E293B")
```

8. Branding: software vs residencia

Uso	Marca que debe aparecer
Login, panel, dashboard, menus internos	GestorCan
Facturas	Residencia cliente
Albaranes	Residencia cliente
Listados impresos	Residencia cliente
Fichas PDF	Residencia cliente
Informes diarios	Residencia cliente

Esta decision es fundamental para el modelo SaaS: GestorCan es el motor del sistema, pero los documentos externos deben tener imagen del negocio que los emite.

9. Logo de residencia

El logo no se guarda como binario en la base de datos. Se guarda una ruta relativa en la columna logo_url. Ejemplo:

```
logo_url = "static/logos/logo.png"
```

El archivo fisico debe existir dentro del proyecto, por ejemplo: gestorcan-backend/static/logos/logo.png. Para evitar errores de ruta se recomienda resolverlo con `os.path.abspath` antes de insertarlo en ReportLab.

```
logo_path = os.path.abspath(residencia.logo_url)

if residencia.logo_url and os.path.exists(logo_path):
    logo_elemento = Image(logo_path)
    logo_elemento.drawWidth = 60
    logo_elemento.drawHeight = 60
else:
    logo_elemento = ""
```

10. Cabecera tipo factura

La cabecera actual se ha alineado como en una factura: datos de la residencia a la izquierda y logo a la derecha, con una linea verde separadora debajo.

```
datos_residencia = Paragraph(
    f"""
    <font size="16"><b>{residencia.nombre}</b></font><br/>
    {residencia.direccion}<br/>
    {residencia.telefono} | {residencia.email}<br/>
    CIF: {residencia.cif}
    """,
    styles["Normal"]
)

tabla_cabecera = Table(
    [[datos_residencia, logo_elemento]],
    colwidths=[360, 140]
)

tabla_cabecera.setStyle(TableStyle([
    ("VALIGN", (0, 0), (-1, -1), "TOP"),
    ("ALIGN", (1, 0), (1, 0), "RIGHT"),
    ("LEFTPADDING", (0, 0), (-1, -1), 0),
    ("RIGHTPADDING", (0, 0), (-1, -1), 0),
    ("BOTTOMPADDING", (0, 0), (-1, -1), 14),
]))

contenido.append(tabla_cabecera)

contenido.append(HRFlowable(
    width="100%",
    thickness=1.5,
    color=colors.HexColor("#3CB371")
))
```

11. Problemas detectados y solucionados

Problema	Causa	Solucion
ModuleNotFoundError: routers	Import incorrecto desde main.py	Usar from app.routers import clientes
NameError: app not defined	include_router antes de app = FastAPI()	Crear app antes de incluir routers
404 en /clientes/1/pdf	Router duplicado o endpoint no registrado	Eliminar segundo router y revisar /docs
NameError: db/residencia	Codigo de BD fuera del endpoint	Mover consulta dentro de la funcion
Logo no aparece	Se leia la primera residencia sin logo	Usar la residencia correcta o limpiar datos antiguos
Logo deformado	Tamaño forzado	Ajustar tamaño o mantener proporcion

12. Estado actual

Comprobacion	Resultado
Servidor local	Arranca con <code>uvicorn app.main:app --reload --port 8080</code>
Swagger	Disponible en <code>/docs</code>
PDF cliente	<code>GET /clientes/1/pdf</code> devuelve PDF
Cabecera	Residencia + direccion + telefono + email + CIF
Logo	Visible en PDF
Mascotas vacias	Muestra mensaje: Este cliente no tiene mascotas registradas

13. Pendientes inmediatos

Prioridad	Tarea	Motivo
Alta	Convertir datos de cliente en tabla visual	Mejorar lectura y apariencia profesional.
Alta	Convertir mascotas en tabla	Preparar ficha para multiples mascotas.
Media	Crear modulo reutilizable de PDF	Evitar repetir cabeceras en facturas, listados e informes.
Media	Relacionar cliente con <code>residencia_id</code>	Base para multi-residencia real.
Media	Evitar guardar PDFs temporales innecesarios	En SaaS conviene generar en memoria o limpiar archivos.
Baja	Endpoint de prueba <code>/residencias/test</code>	Eliminarlo o convertirlo en CRUD real antes de produccion.

14. Siguiete sesion recomendada

El siguiente paso recomendado es convertir el PDF actual en una plantilla visual reutilizable. Primero se debe mejorar la ficha de cliente; despues esa misma estructura servira para facturas, listados e informes.

Prompt sugerido:

Seguimos con GestorCan. Ya tengo funcionando `GET /clientes/{id}/pdf` con ReportLab, cabecera tipo factura, logo de residencia, datos del cliente y mascotas.

Quiero mejorar el PDF:

1. Datos del cliente en tabla visual.
 2. Mascotas en tabla estilo factura.
 3. Crear funciones reutilizables para cabecera, separadores y tablas.
 4. Mantener branding de residencia, no de GestorCan.
- Dame codigo directo adaptado a FastAPI + SQLAlchemy + ReportLab.

15. Checklist de verificacion

Paso	Resultado esperado
Abrir <code>/docs</code>	Aparece <code>GET /clientes/{cliente_id}/pdf</code>
Abrir <code>/clientes/1</code>	Devuelve datos del cliente

Abrir /clientes/1/pdf	Devuelve PDF, no null
Revisar consola	No hay NameError ni rutas duplicadas
Revisar PDF	Cabecera de residencia visible y logo correcto

Documento generado como continuacion de la documentacion tecnica inicial de GestorCan.